Literature Review

Autonomous Driving Lab: Human vs Small LLM vs SAC in MetaDrive

---

1. Introduction to the Research Area

Autonomous driving is one of the most challenging problems in AI, requiring an agent to simultaneously handle perception, planning, and control in dynamic, uncertain environments. Three fundamentally different paradigms have emerged for tackling this problem:

1. **Reinforcement Learning (RL)** — learning to drive through reward signals in simulation
2. **LLM-Based Planning** — using language models to reason semantically about driving scenes
3. **Human Control** — direct keyboard/joystick input as a performance baseline

While each paradigm has been studied independently, no prior work directly compares a small local LLM planner against a trained RL policy and a live human driver on the **exact same procedurally generated driving scenarios** using paired statistical analysis. This project fills that gap.

---

2. Simulation Platform: MetaDrive

Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., & Zhou, B. (2022). *MetaDrive: Composing Diverse Driving Scenarios for Generalizable Reinforcement Learning.* arXiv:2109.12674 — IEEE TPAMI / NeurIPS 2021.

MetaDrive is a lightweight, highly compositional driving simulation platform designed specifically for RL research. Its core feature is **procedural map generation from integer seeds** — a seed deterministically generates a unique road geometry, traffic configuration, and goal location. This makes it ideal for fair multi-agent comparison: every agent in this project faces the exact same map when given the same seed.

Key properties used in this project:

- Seed-controlled procedural generation (train: 1000–1019, test: 2000–2019)
- Supports manual keyboard control (for human agent) and programmatic `step()` (for LLM and RL agents)
- Provides terminal signals: `arrive_dest`, `crash`, `out_of_road`
- Lightweight — runs on CPU without GPU hardware
- 50 Hz simulation step (0.02 s per step), `traffic_density=0.1`, `target_speed=40 km/h`

As Li et al. show, increasing the diversity of training environments through procedural generation improves RL generalization. This project uses MetaDrive as a "neutral experimental laboratory" — the same environment for all three agents, ensuring no agent has a structural advantage.

---

3. Reinforcement Learning for Driving: Soft Actor-Critic (SAC)

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.* arXiv:1801.01290 — ICML 2018.

SAC is the RL algorithm used to train the driving policy in this project. It addresses two key limitations of prior deep RL methods:

1. **High sample complexity** — SAC is off-policy, reusing past experience from a replay buffer (50,000 transitions), making it significantly more sample-efficient than on-policy methods like PPO.
2. **Brittle convergence** — SAC uses a maximum entropy framework: the agent maximizes expected reward while also maximizing action entropy (acting as randomly as possible while still succeeding). This improves stability and robustness across random seeds.

SAC is the standard choice for continuous control tasks. In this project, the action space is `[steering, throttle_brake] ∈ [-1, 1]²` — a continuous 2D action that SAC handles naturally without discretization.

Configuration used in this project:

- Policy: MlpPolicy (fully connected network)
- Learning rate: 3×10^{-4}
- Replay buffer: 50,000 transitions
- Batch size: 256
- Discount factor γ: 0.99
- TensorBoard logging enabled
- Training: 15,000 steps (target: 50,000)
- Eval rewards at checkpoints: 356 (5k) → 1,489 (10k) → 1,293 (15k steps)

The reward improvement from 5k to 10k steps (×4 increase) demonstrates that the agent was actively learning lane-keeping and speed compliance behavior through the shaped reward signal.

---

4. Reward Shaping in RL

Ng, A. Y., Harada, D., & Russell, S. (1999). *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping.* Proceedings of ICML 1999.

A naive sparse reward (only +1 for reaching the goal) provides almost no learning signal in a driving simulator — episodes end quickly with crashes before the agent ever sees a positive reward. Reward shaping adds dense intermediate rewards that guide the agent toward good behavior without changing the optimal policy.

This project implements a multi-objective shaped reward:

```
R = w_progress × R_progress
  + w_lane     × R_lane
  + w_speed    × R_speed
  + w_safety   × R_safety
  + w_task     × R_task
```

Key design decisions motivated by the literature:

Component	Design	Reason
Lane penalty	`–	deviation
Safety penalty	crash/off-road = **−10**	Original −2 was too weak; agent learned to ignore road boundaries
Progress reward	`speed/target + route_progress × 2.0`	Encourages forward movement toward the goal
Smoothness penalty	`-0.1 × steering²`	Discourages jerky steering — improves ride quality metric
Task reward	`+5.0` on arrival	Terminal goal signal

All weights are stored in `config.yaml` and loaded at runtime — making the reward fully configurable without code changes.

---

5. LLM as a High-Level Driving Planner

5.1 Grounding Language in Action: SayCan

Ahn, M., Brohan, A., Brown, N., et al. (2022). *Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan).* arXiv:2204.01691 — Google Robotics.

SayCan is the foundational work that established the **LLM + low-level skill** architecture used in this project. The core insight is that LLMs encode rich semantic knowledge about tasks and procedures, but lack real-world grounding. By combining an LLM (for high-level reasoning and task decomposition) with pre-trained low-level skills (for physical execution), the system achieves both interpretability and physical competence.

In this project, the same hierarchy is applied to driving:

- **High level (LLM):** Receives a text description of the driving state → outputs a JSON maneuver decision at ~1 Hz
- **Low level (PD controller):** Receives the maneuver → computes continuous `[steering, throttle_brake]` at 50 Hz

This separation makes the LLM agent interpretable — we always know *why* it is taking an action (the JSON maneuver is human-readable), unlike the black-box SAC policy.

5.2 LLM for Motion Planning: GPT-Driver

Mao, J., Qian, Y., Ye, Y., & Wang, Y. (2023). *GPT-Driver: Learning to Drive with GPT.* arXiv:2310.01415 — NeurIPS 2023 Workshop on Foundation Models for Decision Making.

GPT-Driver reformulates motion planning as a language modeling problem — the planner inputs and outputs are represented as language tokens, and GPT-3.5 generates driving trajectory coordinates through natural language. Evaluated on the large-scale nuScenes dataset, this approach demonstrates that LLMs' reasoning and generalization capabilities can be directly applied to autonomous driving.

This project follows the same principle but at a smaller scale: instead of GPT-3.5, it uses **qwen2.5:7b** (7 billion parameters, runs locally via Ollama, no internet/cloud required). Instead of trajectory coordinates, it outputs a structured JSON maneuver from a vocabulary of 9 actions. This tests whether the GPT-Driver concept scales down to a small, offline model.

5.3 LLM + Controller Integration: LanguageMPC

Sha, H., Mu, Y., Jiang, Y., Chen, L., Xu, C., Luo, P., & Li, H. (2023). *LanguageMPC: Large Language Models as Decision Makers for Autonomous Driving.* arXiv:2310.03026.

LanguageMPC directly integrates LLM decisions with low-level controllers by guided parameter matrix adaptation. It demonstrates that LLMs can handle complex driving behaviors including multi-vehicle coordination, benefiting from commonsense reasoning capabilities that purely data-driven methods lack.

Key relevance to this project:

- Validates the LLM-as-planner approach for autonomous driving
- Shows that LLM decisions can be "seamlessly integrated with low-level controllers" — the core mechanism used here
- Demonstrates generalizability to rare/complex scenarios through commonsense reasoning

This project's LLM agent uses the same philosophy: the LLM handles the "what to do" (semantic decision), and the deterministic PD controller handles the "how to do it" (physical execution).

---

6. Human Baseline in Driving Research

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). *CARLA: An Open Urban Driving Simulator.* arXiv:1711.03938 — CoRL 2017.

CARLA is the most widely used urban driving simulator, featuring human expert demonstrations for imitation learning benchmarks. However, CARLA benchmarks typically use recorded expert trajectories rather than live human control — and they do not directly compare RL vs LLM vs live human on the same scenarios.

This project uses **live human keyboard control** as the baseline — a human plays the game in real-time on each test seed, and their telemetry is recorded. This provides:

- A genuine human performance ceiling in the same environment
- A comparison point that RL/LLM results must be interpreted against
- Evidence of whether RL "good reward" translates to "actually drives like a human"

Most RL papers omit this comparison entirely, which makes it difficult to assess whether high simulation reward corresponds to real-world driving quality.

---

7. Statistical Evaluation Framework

Demšar, J. (2006). *Statistical Comparisons of Classifiers over Multiple Data Sets.* Journal of Machine Learning Research, 7, 1–30.

Demšar's paper establishes best practices for statistically comparing ML systems across multiple test instances — directly applicable to this project's seed-matched evaluation design.

Why paired statistics are essential: Different seeds generate roads of different difficulty. A harder road will make any agent look worse, regardless of the agent's actual capability. By running all three agents on the same seed and comparing within each seed (paired comparison), road difficulty is eliminated as a confounding variable.

Statistical tests used:

Test	Type	Use case
Paired t-test (`scipy.stats.ttest_rel`)	Parametric	When per-seed differences are approximately normally distributed
Wilcoxon signed-rank (`scipy.stats.wilcoxon`)	Non-parametric	When distribution of differences is unknown or skewed
Cohen's d	Effect size	Measures practical significance (not just statistical significance)

For each metric (RMSE lane deviation, MAE speed error, steering smoothness, completion time), all three pairwise comparisons are tested: RL vs LLM, RL vs Human, LLM vs Human.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations.* Journal of Machine Learning Research, 22(268), 1–8. arXiv:2005.05719.

Stable-Baselines3 (SB3) provides the SAC implementation used in this project. SB3 is the standard RL library for research — it provides tested, reproducible implementations of modern RL algorithms including SAC, PPO, TD3, and DDPG. Using SB3 ensures the SAC baseline is comparable to published results.

---

8. Summary Table

Paper	Year	Contribution to This Project
Li et al. — MetaDrive	2022	Simulation platform, seed-based procedural generation
Haarnoja et al. — SAC	2018	RL algorithm for continuous driving control
Ng et al. — Reward Shaping	1999	Theoretical basis for multi-objective shaped reward
Ahn et al. — SayCan	2022	LLM + low-level skill hierarchy architecture
Mao et al. — GPT-Driver	2023	LLM as motion planner for autonomous driving
Sha et al. — LanguageMPC	2023	LLM integrated with low-level controller for driving
Dosovitskiy et al. — CARLA	2017	Human baseline context in driving simulators
Demšar — Statistical Tests	2006	Paired comparison methodology
Raffin et al. — SB3	2021	SAC implementation library

---

9. Research Gap

Based on this review, the following gap exists in the literature:

> **No prior work directly compares a small (≤7B), locally-running LLM hierarchical planner against a trained SAC RL policy and a live human driver on identical seed-matched procedural driving scenarios, with paired statistical analysis.**

Specifically:

- GPT-Driver uses GPT-3.5 (cloud, large) — not a small offline model
- LanguageMPC uses LLMs for parameter adaptation — not explicit JSON maneuver decisions
- MetaDrive benchmarks compare RL agents — not LLMs or humans
- CARLA benchmarks use recorded human data — not live interactive comparison
- None apply paired t-test / Wilcoxon per-seed comparison across all three paradigms

This project directly addresses this gap with a reproducible, open-source framework.

References

1. Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., & Zhou, B. (2022). MetaDrive: Composing Diverse Driving Scenarios for Generalizable Reinforcement Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. arXiv:2109.12674.

2. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *Proceedings of ICML 2018*. arXiv:1801.01290.

3. Ng, A. Y., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *Proceedings of ICML 1999*, 278–287.

4. Ahn, M., Brohan, A., Brown, N., et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv:2204.01691*.

5. Mao, J., Qian, Y., Ye, Y., & Wang, Y. (2023). GPT-Driver: Learning to Drive with GPT. *NeurIPS 2023 Workshop on Foundation Models for Decision Making*. arXiv:2310.01415.

6. Sha, H., Mu, Y., Jiang, Y., Chen, L., Xu, C., Luo, P., & Li, H. (2023). LanguageMPC: Large Language Models as Decision Makers for Autonomous Driving. *arXiv:2310.03026*.

7. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An Open Urban Driving Simulator. *Proceedings of CoRL 2017*. arXiv:1711.03938.

8. Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1–30.

9. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8. arXiv:2005.05719.